

Mentor / Student Simulator

Техническая документация: архитектура, промпты, отладка — курс Portfolio Investing

1. Архитектура

Система разыгрывает последовательный диалог между двумя LLM-агентами — Mentor (ведёт курс из 10 уроков) и Student (проходит курс, с заданным характером). Оба агента — вызовы одной и той же модели с разными системными промптами; третий агент не используется. Ключевой принцип: вся бухгалтерия курса (номер урока, переход дальше, блафы, завершение) ведётся кодом на Python, а не текстом модели. Модель отвечает только за содержание реплики.

1.1. DialogueState — единственный источник правды

Объект состояния хранит весь прогресс диалога между ходами:

```
current_lesson      # какой урок сейчас (1..10)
turns_on_current_lesson # сколько попыток потрачено на текущий урок
bluffs_scheduled   # список уроков, где студент должен блефовать
bluffs_triggered_on  # список уроков, где блеф уже произошёл
bluff_pending_admission # флаг: следующий ход студента = признание
course_complete     # единственный флаг остановки главного цикла
transcript          # вся история реплик
```

1.2. Главный цикл диалога

Цикл крутится "ментор → студент → ментор → ...", пока `course_complete == False`. На ходу ментора код вызывает модель, парсит служебную STATE-метку из ответа и обновляет `current_lesson`. На ходу студента код заранее (до вызова модели) решает, нужен ли сейчас блеф или признание, и добавляет разовую инструкцию в промпт. Курс завершается только когда `current_lesson == 10 AND advance == true` — это проверка кода, текст модели на это решение не влияет.

1.3. Планирование блафов (детерминированно, в коде)

Расписание блафов считается один раз, до начала диалога. Для явно заданного профиля (Nikolay) — жёсткий список уроков [3, 6, 8]. Для профиля "блефуй N раз, реши сам" (Alex) — код случайным образом выбирает N уникальных уроков (`random.Random(seed).sample(...)`). На каждом ходу студента код проверяет текущий урок против этого списка и добавляет в промпт одноразовую инструкцию "соври" или "признайся" — модели не нужно ничего запоминать самой.

1.4. STATE-метка и парсинг прогресса

В конце каждого сообщения ментор обязан вывести строго одну строку:

```
STATE: lesson=<N> advance=<true|false>
```

Код вытаскивает её регулярным выражением, не пытаясь понять смысл остального текста. Если метка не найдена — код один раз просит модель повторить только эту строку (state-repair). Если и это не помогло — безопасный fallback (`advance=false`),

прогресс не теряется и не додумывается.

1.5. Устойчивость к сбоям API

Каждый вызов модели оборачивается ретраями с нарастающей паузой (3, 6, 9, 12, 15 секунд). Если модель не отвечает после всех попыток — код переключается на следующую модель из списка-кандидата (fallback-цепочка), а не останавливает диалог. Полная остановка происходит только если вообще ни одна модель из списка не ответила — и тогда явная ошибка, а не тихая деградация с пустым текстом.

2. Промпты

2.1. Системный промт Ментора

Статичная часть промпта содержит пять обязательных блоков:

- Фиксированная программа курса — список всех 10 тем прямо в промпте с явным запретом заменять, пропускать или изобретать свои темы. Добавлено после того, как модель самовольно взяла другой набор тем.
- Правила проверки — не принимать ответ без цифр, не двигаться дальше при расплывчатости, принимать признание после блефа, лимит попыток на урок.
- Техники — просить конкретику, спрашивать про непонятное, давать Transfer Test (новый сценарий на тот же навык).
- Запрет раннего завершения и синхронизация — FINAL ASSESSMENT и "COURSE COMPLETE" пишутся только после 10-го урока; если текст содержит "COURSE COMPLETE", служебная метка обязана содержать `advance=true`.
- Обязательная STATE-подпись — формат строки в конце каждого сообщения.

Динамическая часть (добавляется на каждый ход функцией `build_mentor_prompt`): текущий номер урока и его название, число уже потраченных попыток, и — если лимит исчерпан — явное указание "обязан перейти дальше сейчас же".

2.2. Системный промт Студента

Статичная часть: имя, возраст, профессия, капитал, цель, характер (это и отличает Nikolay от Alex), правило отвечать честно с реальными цифрами по умолчанию. Для Alex дополнительно вставлена фраза про систематическую мелкую арифметическую ошибку (его фирменная черта — "фантомные сотни").

Самая важная часть — `bluff_clause`, который не сидит в статичном промпте, а вставляется только на конкретный ход:

```
если bluff_now:
    "Сейчас ты ДОЛЖЕН сблефовать: уверенный, но расплывчатый
    ответ без конкретных цифр."
элиф admission_now:
    "В прошлый раз ты сблефовал. Сейчас ОБЯЗАН признаться
    и дать честный ответ с реальными цифрами."
иначе:
    (ничего не добавляется, обычный честный ответ)
```

Это и есть главный архитектурный фикс: раньше блеф был одним большим условием внутри статичного промпта ("блефуй на уроках X, Y, Z, а после уточняющего вопроса признавайся") — маленькая модель такое правило не удерживала на протяжении всего диалога. Теперь это разбито на одноразовые команды, которые код сам подставляет в нужный момент — модели не нужно ничего помнить.

3. Список отлаженных багов

1. Блеф 16 раз вместо 3 (Alex)

Маленькая модель (Gemma) не удерживала сложное условное правило про номера уроков внутри статичного промпта. Решение: планирование блафов перенесено в код (детерминированно, один раз в начале), модель получает только разовую команду на конкретный ход.

2. Парсер показывал 9/10 вместо 10/10

Свободный текстовый "memory-блок" модель писала непоследовательно. Решение: строгая однострочная метка STATE: `lesson=N advance=true|false` с авто-повтором запроса при сбое парсинга и безопасным fallback.

3. Курс завершался раньше 10-го урока

Модель сама решала, когда писать "COURSE COMPLETE". Решение: `course_complete` выставляется только кодом, при условии `current_lesson == num_lessons` И `advance == true`.

4. Диалог зависал на одном уроке (10+ ходов подряд)

У Alex ментор мог бесконечно требовать точности. Решение: жёсткий лимит `MAX_TURNS_PER_LESSON = 4`, код форсирует переход при исчерпании лимита независимо от ответа модели.

5. Ретраи возвращали пустую строку при сбое API

Тихая деградация диалога — пустая реплика "плыла" дальше по истории. Решение: явная ошибка `ModelCallError`, диалог корректно останавливается с сохранением накопленного текста.

6. Deprecated пакет `google.generativeai`

Пакет прекратил поддержку. Решение: миграция на новый SDK `google.genai` (`client.models.generate_content` вместо `GenerativeModel`).

7. Ключ API приходилось задавать вручную на каждый запуск

Решение: чтение ключа из файла `.env` через `python-dotenv` (`load_dotenv()` при импорте).

8. Неверный/недоступный id модели, ошибки 404 и квота = 0

Часть моделей (например, `gemini-2.0-flash-lite-001`) недоступна на бесплатном тарифе ключа. Решение: добавлена утилита `--list-models` / отдельный `list_models.py` для сверки реально доступных моделей через `client.models.list()`.

9. Модель Gemma систематически нестабильна (500/503)

Бесплатный тариф Gemma периодически перегружен на стороне Google. Решение: fallback-цепочка моделей — если основная не отвечает после всех ретраев, код сам переключается на следующую модель из списка (`MODEL_CANDIDATES`).

10. Дублирование FINAL ASSESSMENT / COURSE COMPLETE

Модель писала завершающий текст, но не проставляла `advance=true` в служебной метке — диалог продолжался ещё на 10+ лишних ходов и писал второй FINAL ASSESSMENT. Решение: код форсирует `advance=True`, если raw-текст содержит

"COURSE COMPLETE" и текущий урок — последний.

11. Модель самовольно меняла темы уроков

Вместо заданных 10 тем (Diversification, Risk vs Return, ...) модель один раз выбрала свой набор тем. Решение: полная программа курса вставлена прямо в системный промпт ментора с явным запретом на замену тем.

12. Случайные префиксы "MENTOR:"/"STUDENT:" в начале ответа

Модель иногда путала роли и подписывалась. Решение: санитайзер (регулярное выражение), который обрезает такие префиксы перед сохранением реплики.